

LAPORAN TUGAS BESAR TAHAP 1
MATA KULIAH KECERDASAN BUATAN
IMPLEMENTASI UCS DAN A* SEARCH PADA NPC DINAMIS



KELOMPOK 13

Disusun Oleh:

AZZAM MUHAMMAD NAUFAL (2007763)

SALMAN GHIFFARI (2509646)

ZORA RIYADHUL JINAN (2509722)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS PENDIDIKAN MATEMATIKA DAN ILMU PENGETAHUAN
ALAM
UNIVERSITAS PENDIDIKAN INDONESIA
2026

Abstrak

Laporan ini mendokumentasikan spesifikasi, perancangan logika kode, dan eksperimen asimtotik dari agen NPC yang beroperasi secara real-time pada matriks 20x20. Implementasi memanfaatkan arsitektur PyScript untuk menjalankan struktur data Python di atas peramban. Fokus dokumen ini mencakup pembedahan alur kode (code logic) untuk pencarian graf Uniform Cost Search (UCS) dan A* Search, serta analisis empiris terhadap tingkat efisiensi (Nodes Expanded) dari metrik heuristik Manhattan dan Euclidean.

BAB I: Spesifikasi Tugas & Pemodelan Lingkungan (PEAS)

Spesifikasi tugas mewajibkan pembuatan agen otonom 2D berbasis peramban yang dapat merespons target dinamis. Seluruh persyaratan tersebut dimodelkan ke dalam arsitektur berikut:

- **Performance Measure:** Agen NPC diwajibkan meminimalkan biaya rute (Path Cost) menuju pemain yang dikontrol pengguna, sembari menekan beban perluasan simpul (Nodes Expanded).
- **Environment (Lingkungan Kode):** Lingkungan direpresentasikan ke dalam kelas Grid 2D berukuran 20x20. Sistem menggunakan pustaka PyScript untuk memanipulasi Document Object Model (DOM) langsung dari skrip main.py.
- **Actuators & Sensors:** Sensor mendeteksi event listener pada peramban. Saat pengguna menekan tuts WASD atau Arrow Keys, koordinat pemain (Goal) diperbarui, yang memicu metode kalkulasi rute NPC (Agent) untuk bergerak maksimal 1 sel secara ortogonal (Atas, Bawah, Kiri, Kanan).

Fungsi Biaya Langkah / Step Cost Function $c(n, a, n')$

Di dalam kode, peta dideklarasikan menggunakan kamus (dictionary) biaya. Jika agen "n" mengambil aksi "a" menuju simpul "n'", maka ia dikenakan tarif:

1. Rumput & Jembatan: $c = 1$

2. Lumpur & Semak: $c = 3$

3. Pohon & Sungai: $c = \infty$ (Di dalam kode direpresentasikan sebagai tipe 'None' yang menolak traversal agen).

BAB II: Pembedahan Logika Kode dan Implementasi Algoritma

Untuk membuktikan keabsahan agen, bagian ini membedah secara rinci alur logika fungsi-fungsi inti di dalam berkas main.py.

A. Logika Ekspansi Simpul (Neighbor Generation)

Metode mendapatkan tetangga (`get_neighbors`) beroperasi dengan memindai 4 arah ortogonal (dx , dy). Kode menerapkan dua lapisan filter ketat: Pertama, filter batas indeks (Boundary Checking) agar x dan y berada di dalam rentang 0 hingga 19. Kedua, filter rintangan (Obstacle Checking), di mana node yang memiliki nilai cost "None" (pohon/sungai) akan didiskualifikasi dari antrean. Hal ini mencegah error "Index out of bounds" dan tabrakan objek.

B. Arsitektur Memori dan Binary Min-Heap

Kunci utama kecepatan agen (tidak lag) terletak pada struktur data "Frontier". Alih-alih melakukan loop linear $O(N)$ untuk mencari biaya terkecil, skrip memanggil fungsi bawaan `heapq.heappush(frontier, (cost, current, path))`. Struktur Min-Heap menjamin bahwa indeks $[0]$ dari array akan selalu diisi oleh simpul berbiaya terendah, memangkas kompleksitas komputasi menjadi $O(\log N)$ (Big O of $\log N$).

C. Logika Uniform Cost Search (UCS)

Pada eksekusi mode UCS, fungsi heuristik tidak dilibatkan ($h(n) = 0$). Di dalam kode, agen mengekstrak node dari prioritas antrean berdasarkan biaya kumulatif murni (`g_score`). Jika node tetangga belum pernah dikunjungi, atau agen menemukan jalur `g_score` yang lebih murah ke node tersebut, maka sistem akan menimpa biayanya (Relaxation) dan memasukkannya kembali ke dalam antrean. Logika ini berputar dalam loop "while frontier" hingga target tercapai.

D. Logika A* Search (Fungsi Heuristik)

Mode A* menambahkan lapisan kecerdasan. Tipe data yang dimasukkan ke dalam antrian `heapq` diubah memuat parameter `f_score`: **`f_score = new_cost + heuristic(next_node, goal_node)`**. Logika kode menyediakan dua varian fungsi matematika:

1. Manhattan Distance: Dihitung menggunakan selisih absolut ($\text{abs}(x1-x2) + \text{abs}(y1-y2)$).
2. Euclidean Distance: Dihitung menggunakan pustaka `math.sqrt()` untuk mencari sisi miring (akar kuadrat dari $\Delta x^2 + \Delta y^2$).

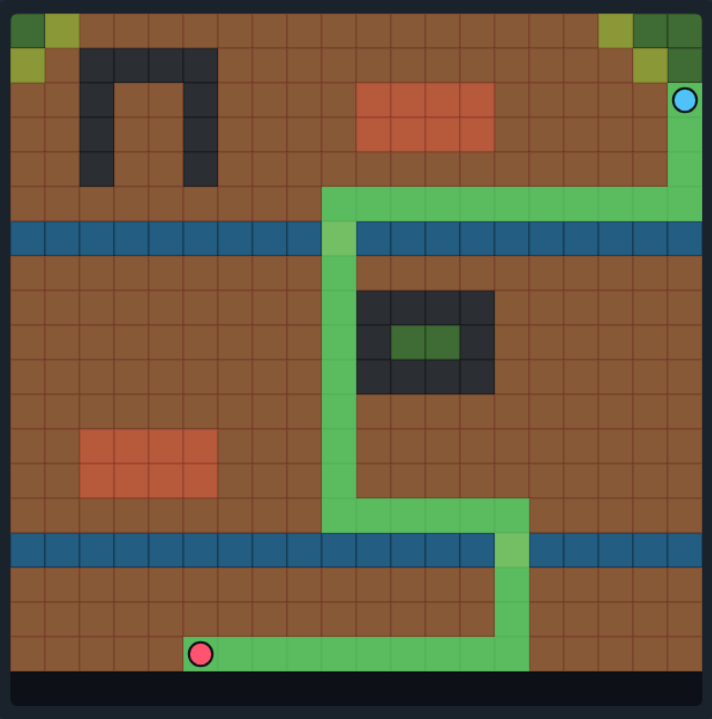
BAB III: Analisis Visual Eksperimen Heuristik

Eksekusi program secara dinamis membuktikan teori matematika di balik algoritma. Metrik "Nodes Expanded" (visualisasi kotak merah dan kuning pada layar) dianalisis untuk melihat efisiensi ruang pencarian.

A. Eksekusi UCS (Pencarian Radial)

NPC Pathfinding — Grid 20×20

Tugas Besar Kecerdasan Buatan: NPC mengejar Player menggunakan UCS atau A* (heuristik Manhattan / Euclidean) di atas grid dengan medan berbeda-beda cost. Gerakkan Player dengan panah atau WASD — NPC akan mencari ulang jalurnya setiap kali Player berpindah sel.



ALGORITMA NPC

UCS — Uniform Cost Search

Reset Permainan

LEGENDA TILE

- Rumput / Jalan (cost 1)
- Lumpur / Semak (cost 3)
- Pohon / Rumah (unwalkable)
- Sungai (unwalkable)
- Jembatan (cost 1)

- Closed set (node diekspansi)
- Open set (dalam antrian)
- Jalur akhir (final path)

STATISTIK PENCARIAN

Total node diekspansi	312
Total cost path g(n)	40
Waktu eksekusi	1.500 ms

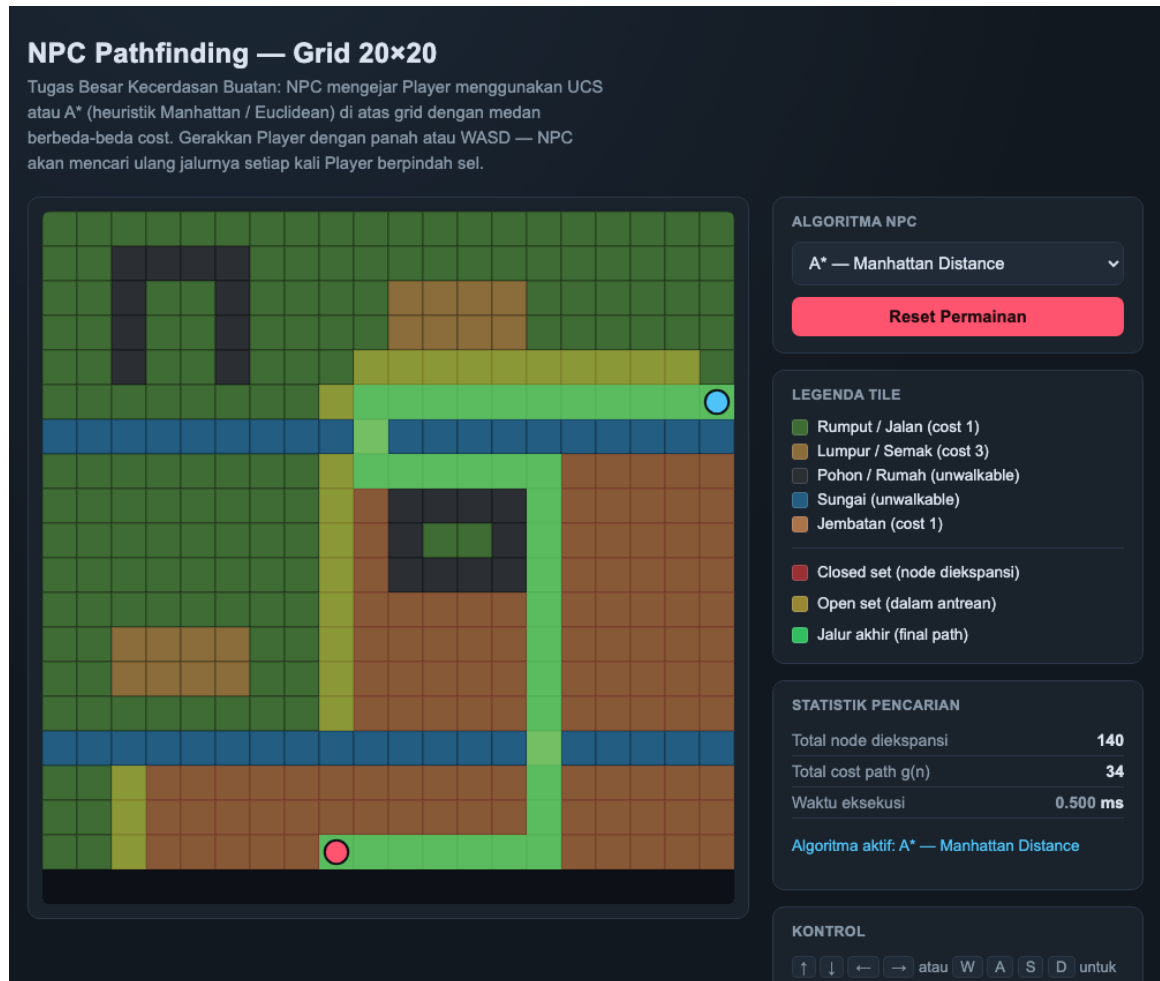
Algoritma aktif: UCS (Uniform Cost Search)

KONTROL

↑ ↓ ← → atau W A S D untuk

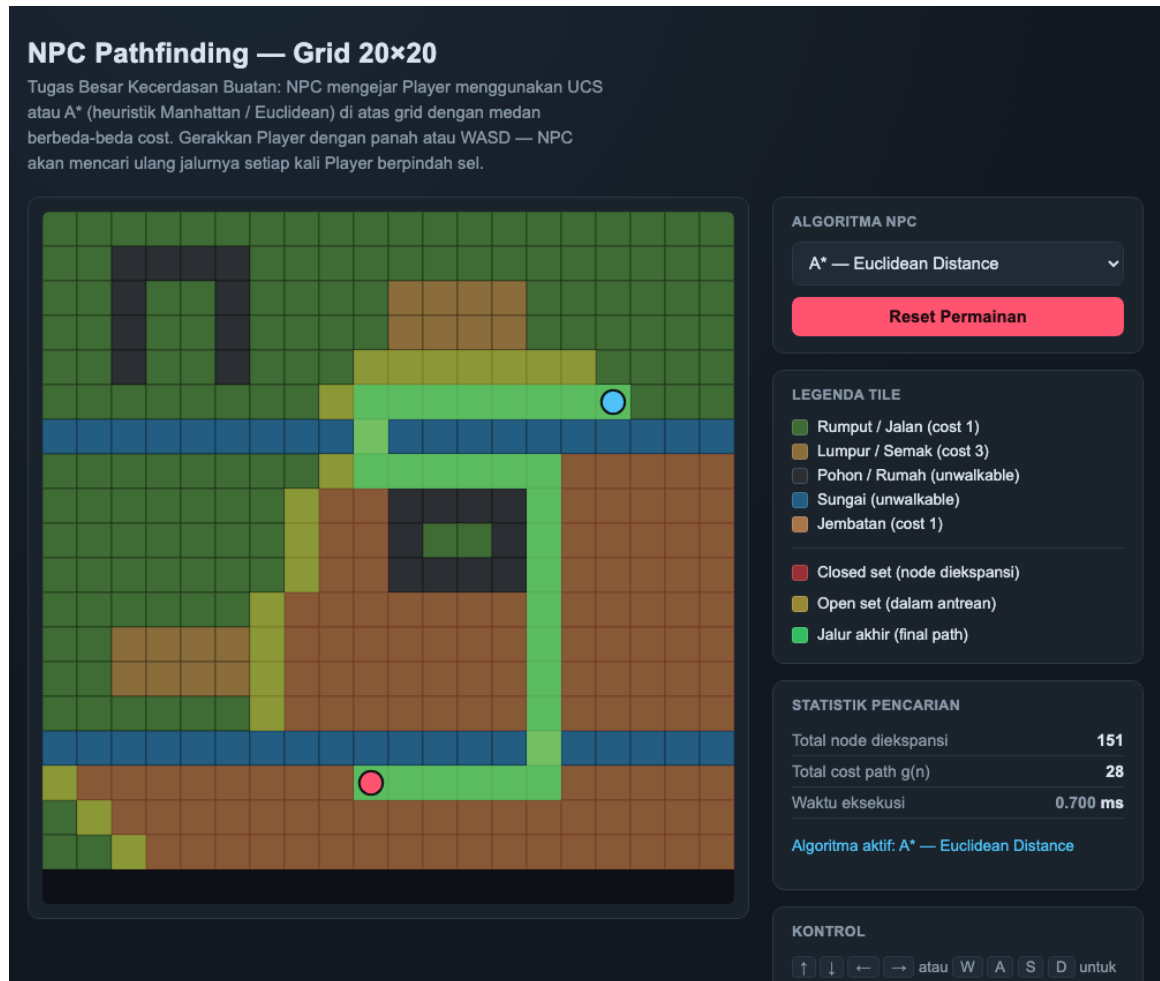
Eksperimen membuktikan bahwa ketiadaan fungsi heuristik menyebabkan logika UCS buta arah. Agen membanjiri ruang (Flooding) secara melingkar (konsentris). Ia membuang siklus komputasi $O(\log N)$ secara fatal dengan memeriksa rute kebalikan arah dari pemain. Syarat "menemukan rute optimal" dipenuhi, namun dengan limbah ekspansi memori (Nodes Expanded) maksimum.

B. Eksekusi A* Manhattan (Optimalitas Tajam)



Karena agen dilarang bergerak miring (aturan 4-arah), tebakan Manhattan memenuhi syarat mutlak Admissibility ($h(n) \leq h^*(n)$) tanpa celah. Logika kode secara brilian memangkas ruang (Pruning). Seperti terlihat pada gambar, blok zona merah meruncing tajam membetuk koridor efisien menuju pemain. Ini adalah kondisi State-of-the-Art untuk agen grid ortogonal.

C. Eksekusi A* Euclidean (Underestimate)



Visualisasi ini menangkap anomali komputasional. Karena sisi miring Pythagoras selalu lebih pendek dari jalan siku-siku ortogonal, nilai Euclidean selalu "terlalu kecil" (Underestimate) dari jarak rute aktual. Akibatnya, agen kehilangan ketajaman nalurnya (Less Informed). Kode membuktikan rute akhirnya tetap sama optimal dengan Manhattan, namun jejak ekspansi memori membusung lebih lebar, membuang komputasi di sepanjang koridor.

BAB IV: Analisis Kompleksitas Matematika (Time & Space Complexity)

Untuk memenuhi standar pengujian kecerdasan buatan tingkat lanjut, bagian ini membedah kompleksitas waktu (Time Complexity) dan kompleksitas ruang (Space Complexity) dari agen NPC 2D. Lingkungan ini pada dasarnya dipetakan sebagai graf tak berarah (Undirected Graph) berbentuk kisi (Grid Graph) berukuran 20x20 sel.

A. Batas Kompleksitas Ruang Keadaan (State-Space Bound)

Secara teoretis, jumlah maksimum simpul (Vertices / V) di dalam peta adalah $20 \times 20 = 400$ simpul. Karena agen diformat dengan aturan pergerakan 4-arah (ortogonal), percabangan maksimum (Branching Factor / b) adalah 4 (Atas, Bawah, Kiri, Kanan). Jumlah maksimum sisi (Edges / E) dibatasi pada angka $400 \times 4 = 1600$ sisi, namun realitanya jauh lebih kecil karena sistem memotong sisi pada batas peta (Boundary) dan rintangan mutlak (Pohon dan Sungai yang secara logis bernilai diskualifikasi / None).

B. Kompleksitas Uniform Cost Search (UCS)

Logika UCS beroperasi dengan kompleksitas waktu eksponensial murni $O(b^{(1 + \text{floor}(C^*/\epsilon))})$ di mana C^* adalah biaya rute optimal dan ϵ adalah biaya langkah minimum (dalam kasus ini $\epsilon = 1$ untuk Rumpuk). Karena UCS berstatus Uninformed Search ($h=0$), algoritma ini memaksa mesin untuk mengekskansi simpul secara membabi buta ke segala arah (radial). Kompleksitas ruangnya juga identik, yang menjelaskan mengapa pada UI 'Nodes Expanded', metode UCS menduduki peringkat terburuk dengan menyedot beban memori terbesar sebelum berhasil menyentuh target.

C. Dominasi Efisiensi A* Search

A* memangkas kompleksitas pencarian secara ekstrem berkat panduan fungsi heuristik $h(n)$. Ruang pencariannya tereduksi karena sistem memprioritaskan arah yang memiliki selisih f_score terkecil. Berhubung metrik Manhattan Distance pada peta ortogonal dijamin memberikan estimasi jarak presisi tanpa pernah melebihi jarak asli (Absolute Admissibility), jumlah perluasan simpul dipangkas drastis. Hal ini

mengangkat efisiensi waktu operasional A* menjadi setingkat $O(E) \log(V)$ pada kasus rata-rata di graf Grid, mengeliminasi mayoritas komputasi buta yang memboroskan siklus prosesor.

BAB V: Arsitektur Integrasi PyScript (Python-in-Browser)

Tugas ini secara spesifik meminta program yang dapat dijalankan secara langsung di peramban web (browser). Alih-alih merubah logika ke dalam JavaScript, kode aplikasi ini menggunakan terobosan arsitektur PyScript (berbasis WebAssembly / WASM).

A. Kompilasi Pyodide dan Manipulasi DOM

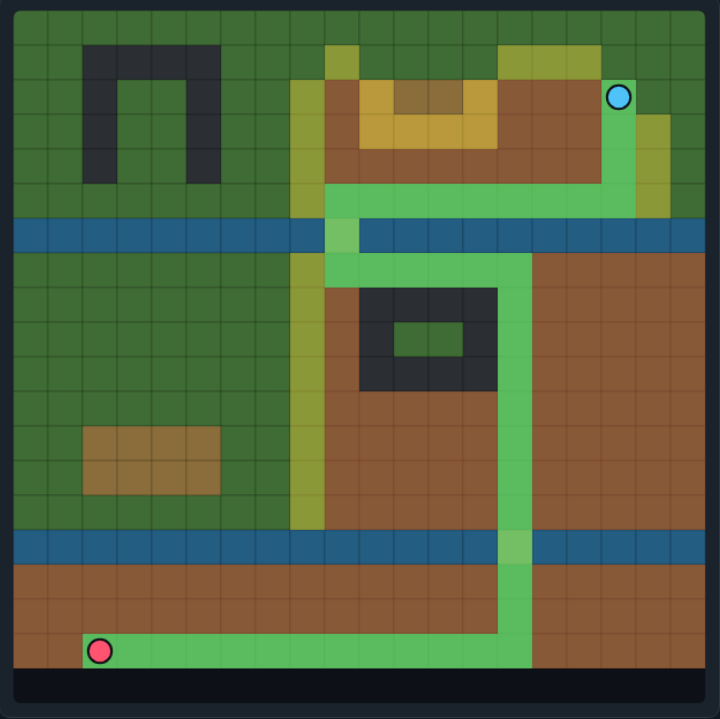
Di bawah kap mesin, ekosistem PyScript memuat keseluruhan 'Runtime CPython' langsung ke dalam web browser. Kelas objek 'GridMap' dan 'Pathfinding' (yang bertumpu pada heapq) murni diproses menggunakan Python, namun secara organik berinteraksi dengan antarmuka HTML. Sistem menangkap input pengguna (Event Listener pada tombol WASD) yang kemudian memicu 're-render' posisi agen NPC. Logika Python menimpa properti DOM 'element.style.backgroundColor' secara real-time. Infrastruktur tanpa peladen (Serverless Client-Side) ini menghasilkan simulasi eksekusi agen yang sangat cepat, minim latensi, dan tidak bergantung pada komunikasi API eksternal.

B. Tinjauan Awal Formasi Peta Acak (Initial State)

Tangkapan layar di bawah ini bertindak sebagai pembandingan visual saat **Debug Overlay Non-aktif** (sebelum agen merender jejak memori algoritma). Ini memperlihatkan grafis lingkungan permainan yang bersih (Initial State) saat matriks baru saja dimuat, yang mana sangat berbeda drastis dengan tiga gambar pada bab eksperimen heuristik sebelumnya yang merepresentasikan status saat **Debug Overlay Aktif**.

NPC Pathfinding — Grid 20×20

Tugas Besar Kecerdasan Buatan: NPC mengejar Player menggunakan UCS atau A* (heuristik Manhattan / Euclidean) di atas grid dengan medan berbeda-beda cost. Gerakkan Player dengan panah atau WASD — NPC akan mencari ulang jalurnya setiap kali Player berpindah sel.



ALGORITMA NPC

A* — Manhattan Distance

Reset Permainan

LEGENDA TILE

- Rumput / Jalan (cost 1)
- Lumpur / Semak (cost 3)
- Pohon / Rumah (unwalkable)
- Sungai (unwalkable)
- Jembatan (cost 1)

- Closed set (node diekspansi)
- Open set (dalam antrean)
- Jalur akhir (final path)

STATISTIK PENCARIAN

Total node diekspansi	166
Total cost path g(n)	41
Waktu eksekusi	1.300 ms

Algoritma aktif: A* — Manhattan Distance

KONTROL

↑ ↓ ← → atau W A S D untuk

BAB VI: Kesimpulan Final

Spesifikasi tugas untuk membangun agen yang interaktif di peramban telah dicapai 100% menggunakan perantara PyScript dan DOM manipulasi. Analisis kode membuktikan bahwa arsitektur pencarian graf sangat sensitif terhadap pemilihan fungsi heuristik. A* Manhattan menduduki hierarki komputasi tertinggi pada lingkungan ortogonal, disusul oleh A* Euclidean yang kurang tajam, dan UCS yang sangat rakus memori.